

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	PRITHIYANKA SHREE M	Reg. No.:	192521144
Section / Batch:	2025	Date:	29-08-2026

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Q1 Bin Packing Approximation Algorithm

100 marks

Write a pseudocode for an approximation algorithm such as First-Fit Decreasing to pack items into bins of fixed capacity. Clearly define inputs (item sizes, bin capacity) and output (set of bins with assigned items). Design an algorithm that first orders items and then places each item into the first suitable bin. Use loops and conditional checks effectively for sorting and bin assignment. Ensure the pseudocode is well-structured, readable, and logically organized. Present all major steps clearly in sequence. Handle all items correctly and produce a valid approximate packing solution.

Code of Conduct

I PRITHIYANKA SHREE M (192521144) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M PRITHIYANKA SHREE

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Write a pseudocode for an approximation algorithm such as First-Fit Decreasing to pack items into bins of fixed capacity. Clearly define inputs (item sizes, bin capacity) and output (set of bins with assigned items). Design an algorithm that first orders items and then places each item into the first suitable bin. Use loops and conditional checks effectively for sorting and bin assignment. Ensure the pseudocode is well-structured, readable, and logically organized. Present all major steps clearly in sequence. Handle all items correctly and produce a valid approximate packing solution.

PSEUDO CODE:

ALGORITHM FirstFitDecreasing(items, binCapacity)

INPUT:

items → List of item sizes

binCapacity → Maximum capacity of each bin

OUTPUT:

bins → Set of bins containing assigned items

BEGIN

// Step 1: Sort items in decreasing order

SORT items in decreasing order

// Step 2: Create an empty set of bins

bins ← empty list

// Step 3: Place each item into the first suitable bin

FOR each item IN items DO

placed ← FALSE

// Check existing bins

FOR each bin IN bins DO

IF (remaining capacity of bin $\geq$ item) THEN

PLACE item into bin

UPDATE remaining capacity of bin

placed ← TRUE

BREAK

END IF

END FOR

```
// Step 4: If no suitable bin is found, create a new bin

IF placed = FALSE THEN

    CREATE a new bin

    PLACE item into the new bin

    ADD new bin to bins

END IF

END FOR

// Step 5: Return the final packing

RETURN bins

END
```

CODE:

Your Code

```
1 n = int(input())
2 capacity = int(input())
3
4 items = list(map(int, input().split()))
5
6 # Sort items in decreasing order using Bubble Sort
7 for i in range(n):
8     for j in range(0, n - i - 1):
9         if items[j] < items[j + 1]:
10             temp = items[j]
11             items[j] = items[j + 1]
12             items[j + 1] = temp
13
14 bins = []
15 remaining = []
16
```

```

17 # First-Fit Decreasing
18 for i in range(n):
19     placed = False
20
21     for j in range(len(bins)):
22         if remaining[j] >= items[i]:
23             bins[j].append(items[i])
24             remaining[j] = remaining[j] - items[i]
25             placed = True
26             break
27
28     if placed == False:
29         new_bin = []
30         new_bin.append(items[i])
31         bins.append(new_bin)
32         remaining.append(capacity - items[i])
33
34 # Output
35 print("Bins:")
36
37 for i in range(len(bins)):
38     print("Bin", i + 1, ":", end=" ")
39
40     for j in range(len(bins[i])):
41         print(bins[i][j], end=" ")
42
43     print()
44

```

INPUT AND OUTPUT:

Input

```

6
10
7 5 6 4 3 2

```

Output

```

Bins:
Bin 1 : 7 3
Bin 2 : 6 4
Bin 3 : 5 2
Number of bins: 3

```

